hedgehog
Home: https://hedgehog.qa/
Hackage: https://hackage.haskell.org/package/hedgehog

PropEr Testing: https://propertesting.com/toc.html
5 tips for better hedgehog testing: https://jacobstanley.io/5-tips-for-better-hedgehog-tests/

```
-- Hedgehog
data Group = Group { groupName :: !GroupName, groupProperties :: !
[(PropertyName, Property)] }
data GroupName
 -- instance IsString e.g. "TestGroup" :: GroupName
data PropertyName
 -- instance IsString e.g. "TestProp" :: PropertyName

check :: MonadIO m => Property -> m Bool
recheck :: MonadIO m => Size -> Seed -> Property -> m ()
recheckAt :: MonadIO m => Seed -> Skip -> Property -> m ()
discover :: TExpQ Group
discoverPrefix :: String -> TExpQ Group
checkParallel :: MonadIO m => Group -> m Bool
checkSequential :: MonadIO m => Group -> m Bool

data Confidence                     -- instance Num e.g. 1 ::
Confidence           default 10^9
data TestLimit                      -- instance Num e.g. 1 ::
TestLimit                default 200
data DiscardLimit                   -- instance Num e.g. 1 ::
DiscardLimit         default 10000
data ShrinkLimit                    -- instance Num e.g. 1 ::
ShrinkLimit          default 1000
data ShrinkRetries                  -- instance Num e.g. 1 ::
ShrinkRetries            default 0
data Skip                           -- instance Num e.g. 1 :: Skip
verifiedTermination :: Property -> Property
withConfidence :: Confidence -> Property -> Property

withTests :: TestLimit -> Property -> Property

withDiscards :: DiscardLimit -> Property -> Property
withShrinks :: ShrinkLimit -> Property -> Property
withRetries :: ShrinkRetries -> Property -> Property
withSkip :: Skip -> Property -> Property
newtype Size = Size { unSize :: Int }
data Seed = Seed { seedValue :: !Word64, seedGamma :: !Word64 }



-- Properties
data Property
data PropertyT m a
instance Monad m => MonadTest PropertyT m
```

```
property :: HasCallStack => PropertyT IO () -> Property
test :: Monad m => TestT m a -> PropertyT m a
forAll :: (Monad m, Show a, HasCallStack) => Gen a -> PropertyT m a
forAllWith :: (Monad m, HasCallStack) => (a -> String) -> Gen a ->
PropertyT m a
discard :: Monad m => PropertyT m a



-- Tests i.e. assertions
type Test = TestT Identity
data TestT m a
instance Monad m => MonadTest (TestT m)

class Monad m => MonadTest m where
  liftTest :: Test a -> m a

annotate :: (MonadTest m, HasCallStack) => String -> m ()
annotateShow :: (MonadTest m, Show a, HasCallStack) => a -> m ()
footnote :: MonadTest m => String -> m ()
footnoteShow :: (MonadTest m, Show a) => a -> m ()
success :: MonadTest m => m ()
failure :: (MonadTest m, HasCallStack) => m a
assert :: (MonadTest m, HasCallStack) => Bool -> m ()
diff :: (MonadTest m, Show a, Show b, HasCallStack) => a -> (a -> b
-> Bool) -> b -> m ()
(===) :: (MonadTest m, Eq a, Show a, HasCallStack) => a -> a -> m
()
(/==) :: (MonadTest m, Eq a, Show a, HasCallStack) => a -> a -> m
()
tripping :: (MonadTest m, Applicative f, Show b, Show (f a), Eq (f
a), HasCallStack) => a -> (a -> b) -> (b -> f a) -> m ()
eval :: (MonadTest m, HasCallStack) => a -> m a
evalNF :: (MonadTest m, NFData a, HasCallStack) => a -> m a
evalM :: (MonadTest m, MonadCatch m, HasCallStack) => m a -> m a
evalIO :: (MonadTest m, MonadIO m, HasCallStack) => IO a -> m a
evalEither :: (MonadTest m, Show x, HasCallStack) => Either x a ->
m a
evalEitherM :: (MonadTest m, Show x, MonadCatch m, HasCallStack) =>
m (Either x a) -> m a
evalExceptT :: (MonadTest m, Show x, HasCallStack) => ExceptT x m a
-> m a
evalMaybe :: (MonadTest m, Show a, HasCallStack) => Maybe a -> m a
evalMaybeM :: (MonadTest m, Show a, MonadCatch m, HasCallStack) =>
m (Maybe a) -> m a



-- Coverage
data LabelName                                  -- instance IsString
e.g. "x" :: LabelName

classify :: (MonadTest m, HasCallStack) => LabelName -> Bool -> m
()
```

```
cover :: (MonadTest m, HasCallStack) => CoverPercentage ->
LabelName -> Bool -> m ()
label :: (MonadTest m, HasCallStack) => LabelName -> m ()
collect :: (MonadTest m, Show a, HasCallStack) => a -> m ()



… etc …
```

Hedgehog.Gen

```
type Gen = GentT Identity
data GenT m a
instance Monad m => MonadGen (GenT m)

class (Monad m, Monad (GenBase m)) => MonadGen m where
  type GenBase m :: Type -> Type
  toGenT :: m a -> GenT (GenBase m) a
  fromGenT :: GenT (GenBase m) a -> m a

-- Sampling
sample :: MonadIO m => Gen a -> m
a                                          -- size 30
print :: (MonadIO m, Show a) => Gen a -> m ()
printTree :: (MonadIO m, Show a) => Gen a -> m ()
printWith :: (MonadIO m, Show a) => Size -> Seed -> Gen a -> m ()
printTreeWith :: (MonadIO m, Show a) => Size -> Seed -> Gen a -> m
()



-- Shrink
shrink :: MonadGen m => (a -> [a]) -> m a -> m a
prune :: MonadGen m => m a -> m a

-- Resizing
small :: MonadGen m => m a -> m a
scale :: MonadGen m => (Size -> Size) -> m a -> m a
resize :: MonadGen m => Size -> m a -> m a
sized :: MonadGen m => (Size -> m a) -> m a



-- Generators
integral :: forall m a. (MonadGen m, Integral a) => Range a -> m a
integral_ :: (MonadGen m, Integral a) => Range a -> m a
int :: MonadGen m => Range Int -> m Int
int8 :: MonadGen m => Range Int8 -> m Int8
int16 :: MonadGen m => Range Int16 -> m Int16
int32 :: MonadGen m => Range Int32 -> m Int32
int64 :: MonadGen m => Range Int64 -> m Int64
word :: MonadGen m => Range Word -> m Word
word8 :: MonadGen m => Range Word8 -> m Word8
word16 :: MonadGen m => Range Word16 -> m Word16
word32 :: MonadGen m => Range Word32 -> m Word32
```

```
word64 :: MonadGen m => Range Word64 -> m Word64
realFloat :: (MonadGen m, RealFloat a) => Range a -> m a
realFrac_ :: (MonadGen m, RealFrac a) => Range a -> m a
float :: MonadGen m => Range Float -> m Float
double :: MonadGen m => Range Double -> m Double
enum :: (MonadGen m, Enum a) => a -> a -> m a
enumBounded :: (MonadGen m, Enum a, Bounded a) => m a
bool :: MonadGen m => m Bool
bool_ :: MonadGen m => m Bool
binit :: MonadGen m => m Char
octit :: MonadGen m => m Char
digit :: MonadGen m => m Char
hexit :: MonadGen m => m Char
lower :: MonadGen m => m Char
upper :: MonadGen m => m Char
alpha :: MonadGen m => m Char
alphaNum :: MonadGen m => m Char
ascii :: MonadGen m => m Char
latin1 :: MonadGen m => m Char
unicode :: MonadGen m => m Char
unicodeAll :: MonadGen m => m Char
string :: MonadGen m => Range Int -> m Char -> m String
text :: MonadGen m => Range Int -> m Char -> m Text
utf8 :: MonadGen m => Range Int -> m Char -> m ByteString
bytes :: MonadGen m => Range Int -> m ByteString

-- Transforms
constant :: MonadGen m => a -> m a
element :: (Foldable f, MonadGen m) => f a -> m a
element_ :: MonadGen m => [a] -> m a
choice :: MonadGen m => [m a] -> m a
frequency :: MonadGen m => [(Int, m a)] -> m a
recursive :: MonadGen m => ([m a] -> m a) -> [m a] -> [m a] -> m
a                     -- recursive f [<non recursive>]
[<recursive>]
discard :: MonadGen m => m a
filter :: (MonadGen m, GenBase m ~ Identity) => (a -> Bool) -> m a
-> m a
mapMaybe :: (MonadGen m, GenBase m ~ Identity) => (a -> Maybe b) ->
m a -> m b
filterT :: MonadGen m => (a -> Bool) -> m a -> m a
mapMaybeT :: MonadGen m => (a -> Maybe b) -> m a -> m b
just :: (MonadGen m, GenBase m ~ Identity) => m (Maybe a) -> m a
justT :: MonadGen m => m (Maybe a) -> m a
maybe :: MonadGen m => m a -> m (Maybe a)
                     -- sometimes Nothing
either :: MonadGen m => m a -> m b -> m (Either a b)
                     -- more Right than Left
either_ :: MonadGen m => m a -> m b -> m (Either a b)
                     -- equal
list :: MonadGen m => Range Int -> m a -> m [a]
seq :: MonadGen m => Range Int -> m a -> m (Seq a)
nonEmpty :: MonadGen m => Range Int -> m a -> m (NonEmpty a)
```

```
set :: (MonadGen m, Ord a) => Range Int -> m a -> m (Set a)
map :: (MonadGen m, Ord k) => Range Int -> m (k, v) -> m (Map k v)
freeze :: MonadGen m => m a -> m (a, m a)
subterm :: MonadGen m => m a -> (a -> a) -> m a
subtermM :: MonadGen m => m a -> (a -> m a) -> m a
subterm2 :: MonadGen m => m a -> m a -> (a -> a -> a) -> m a
subtermM2 :: MonadGen m => m a -> m a -> (a -> a -> m a) -> m a
subterm3 :: MonadGen m => m a -> m a -> m a -> (a -> a -> a -> a)
-> m a
subtermM3 :: MonadGen m => m a -> m a -> m a -> (a -> a -> a -> m
a) -> m a
subsequence :: MonadGen m => [a] -> m [a]
subset :: MonadGen m => Set a -> m (Set a)
shuffle :: MonadGen m => [a] -> m [a]
shuffleSeq :: MonadGen m => Seq a -> m (Seq a)

…
```

Hedgehog.Range

```
newtype Size = Size { unSize::Int }
data Range a
origin :: Range a -> a
bounds :: Size -> Range a -> (a,a)
lowerBound :: Ord a => Size -> Range a -> a
upperBound :: Ord a => Size -> Range a -> a

singleton :: a -> Range a
constant :: a -> a -> Range a
      -- constantFrom x x y
constantFrom :: a -> a -> a -> Range a
      -- constantFrom <origin> <lower> <upper>
constantBounded :: (Bounded a, Num a) => Range a
linear :: Integral a => a -> a -> Range a
      -- linearFrom x x y
linearFrom :: Integral a => a -> a -> a -> Range a
         -- linearFrom <origin> <lower> <upper>
linearFrac :: (Fractional a, Ord a) => a -> a -> Range a
linearFracFrom :: (Fractional a, Ord a) => a -> a -> a -> Range a
exponential :: Integral a => a -> a -> Range a
exponentialFrom :: Integral a => a -> a -> a -> Range a
exponentialFrac :: (Fractional a, Ord a) => a -> a -> Range a
exponentialFracFrom :: (Fractional a, Ord a) => a -> a -> a ->
Range a
```

**Property**
A property to be tested.
Properties may be **grouped** together.

**Generator**

Generates a random value chosen from a **range** of values.
Random value is generated from a **seed**.

```
import Hedgehog.Gen
import Hedgehog.Range
import Hedgehog.Internal.Seed

gen = int (linear 1 100)
printWith 99 (Seed 1 1) gen             -- fixed seed
random >>= \s -> printWith 99 s gen     -- random seed
```

3 types of ranges:
- **constant** range: all **equal** distribution
- **linear** range: linear scale
- **exponential** range: exponential scale

A range is associated with:
- **lower/upper** bound
- **origin** (linear, exponential only): center of distribution
  close to origin **high** distribution
  far from origin **low** distribution
- **size (0~99)**: scale of the bounds
  size 0 min scale i.e. lower == upper bound
  size 99 max scale i.e. any value in [lower,upper]
  size > 99 size (mod 100)

In a test, begins from **size 0** and **incremented** per test run (https://hackage.haskell.org/package/hedgehog-1.2/docs/src/Hedgehog.Internal.Runner.html#checkReport)

e.g. range [0, 100] with different range types

```
-- constant range [0,100]
[0,10]   11% ■■▏ ··················
[10,20]  11% ■■▏ ··················
[20,30]   8% ■▏ ···················
[30,40]   9% ■▏ ···················
[40,50]  11% ■■▏ ··················
[50,60]  10% ■■ ···················
[60,70]  14% ■■▍ ··················
[70,80]   6% ▌ ····················
[80,90]  12% ■■▏ ··················
[90,100]  8% ■▏ ···················

-- linear range [0,100] with origin 0
[0,10]   36% ■■■■■■■▏ ·····------
[10,20]  22% ■■■■▏ ···------------
[20,30]  16% ■■■▏ ··--------------
[30,40]  11% ■■▏ ··---------------
[40,50]   6% ▌ ···---------------
[50,60]   6% ▌ ···---------------
[60,70]   2% ▏ ···---------------
[70,80]   0% ·····---------------
[80,90]   1% | ···---------------
[90,100]  0% ·····---------------
```

```
-- exponential range [0,100] with origin 0
[0,10]   68% ▉▉▉▉▉▉▉▉▉▉▉▉▉▏·····
[10,20]  12% ▉▉▏···············
[20,30]   6% ▉▏···············
[30,40]   3% ▏···············
[40,50]   5% ▉···············
[50,60]   0% ················
[60,70]   3% ▏···············
[70,80]   2% ▏···············
[80,90]   0% ················
[90,100]  1% |···············
```

e.g. different origin/size

```
-- linear range [0, 100] with origin=0 size=99
-- i.e. range = linear 0 100
[0,10]   36% ▉▉▉▉▉▉▉▏··········
[10,20]  22% ▉▉▉▉▏·············
[20,30]  16% ▉▉▉▏··············
[30,40]  11% ▉▉▏···············
[40,50]   6% ▉▏···············
[50,60]   6% ▉▏···············
[60,70]   2% ▏···············
[70,80]   0% ················
[80,90]   1% |···············
[90,100]  0% ················

-- linear range [0,100] with origin=50 size=99
-- i.e. range = linear 50 0 100
[0,10]    0% ················
[10,20]   7% ▉▏···············
[20,30]   9% ▉▉···············
[30,40]  19% ▉▉▉▉·············
[40,50]  30% ▉▉▉▉▉▉▉···········
[50,60]  19% ▉▉▉▉·············
[60,70]  11% ▉▉▏··············
[70,80]   4% ▉···············
[80,90]   1% |···············
[90,100]  0% ················

-- linear range [0,100] with origin=0 size=30
-- i.e. range = resize 30 $ linear 0 100
[0,10]   33% ▉▉▉▉▉▉▏··········
[10,20]  33% ▉▉▉▉▉▉▏··········
[20,30]  30% ▉▉▉▉▉▉···········
[30,40]   4% ▉···············
[40,50]   0% ················
[50,60]   0% ················
[60,70]   0% ················
[70,80]   0% ················
[80,90]   0% ················
[90,100]  0% ················
```

```
-- linear range [0,100] with origin=50 size=30
-- i.e. range = resize 30 $ linearFrom 50 0 100
[0,10]    0% ····················
[10,20]   0% ····················
[20,30]   0% ····················
[30,40]  18% ███▌··············
[40,50]  32% ██████▌···········
[50,60]  33% ██████▌···········
[60,70]  17% ███▌··············
[70,80]   0% ····················
[80,90]   0% ····················
[90,100]  0% ····················
```

Code for above

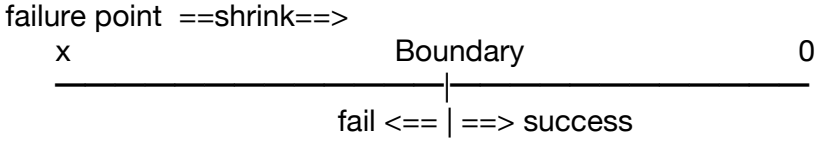
```
import Data.Foldable
import GHC.Exts (fromString)
import Hedgehog
import Hedgehog.Gen
import Hedgehog.Range

testCoverage :: Gen Int -> Property
testCoverage gen = property $ forAll gen >>= printCoverages

printCoverages :: MonadTest m => Int -> m ()
printCoverages x = traverse_ coverIn $ enumFromThenTo 0 step 90
  where
    step = 10
    coverIn lb = let ub = lb + step in classify ("[" <> fromString
(show lb) <> "," <> fromString (show ub) <> "]") $ x > lb && x <=
ub

-- e.g. coverage of varying origin/size
testCoverage $ int $ linear 0 100
testCoverage $ int (linearFrom 50 0 100)
testCoverage $ resize 30 $ int $ linear 0 100
testCoverage $ resize 30 $ int $ linearFrom 50 0 100
-- range types
testCoverage $ int $ constant 0 100
testCoverage $ int $ exponential 0 100
```

**Shrink**
A mechanism to try to specify the **boundary case** when a test fails.
When a test fails at some value**,** it **shrinks** (i.e. **reduces**) the test value from the failure point until it reaches the **success/fail boundary**



e.g. determine first integer failing x <= 200

```
prop_shrinktest :: Property
prop_shrinktest = property $ do
  x <- forAll $ int $ linear 0 1000
  (x <= 200) === True
```

Finds boundary at 201 via shrinking

```
  ✗ prop_shrinktest failed at /Users/yuinishizawa/Projects/
playground/haskell/haskellInDepthGHC9/src/scratch/hedgehog.hs:56:14
    after 26 tests and 4 shrinks.
       ┏━━ src/scratch/hedgehog.hs ━━━
    53 ┃ prop_shrinktest :: Property
    54 ┃ prop_shrinktest = property $ do
    55 ┃   x <- forAll $ int $ linear 0 1000
       ┃   | 201
    56 ┃   (x <= 200) === True
       ┃   ^^^^^^^^^^^^^^^^^^
       ┃   | ━━━ Failed (- lhs) (+ rhs) ━━━
       ┃   | - False
       ┃   | + True
```

Basic

```
-- Assertions
--
1 === 1
1 /== 1

-- tripping: test 1-1 conversions
-- Shows better error message
tripping :: (MonadTest m, Applicative f, Show b, Show (f a), Eq (f
a), HasCallStack) =>
  a -> (a -> b) -> (b -> f a) -> m ()
let readMaybe :: String -> Either String Int
    readMaybe s = case reads s of [(a, "")] -> Right (a + 1); _ ->
Left "ERROR"
tripping 10 show readMaybe
readMaybe (show 10) === pure 10            -- analogous with ===
```

```
# Error with tripping
    114 ┃         tripping 10 show readMaybe
        ┃         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
        ┃         | ━━━ Intermediate ━━━
        ┃         | "1"
        ┃         | ━━━ - Original) (+ Roundtrip ━━━
        ┃         | - Right 10
        ┃         | + Right 11
```

```
# Error without tripping
    114 |         readMaybe (show 10) === pure 10
        |         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
        |         | ━━ Failed (- lhs) (+ rhs) ━━
        |         | - Right 11
        |         | + Right 10
```

Generator

```
import Hedgehog.Gen as Gen
import Hedgehog.Range as Range

int (G.constant 1 100)

-- maybe
G.maybe :: MonadGen m => m a -> m (Maybe a)
x :: Gen a
G.maybe x :: Gen (Maybe a)        -- Nothing : Just = 2 : 1 + size
i.e. Just most of the time
resize 1 $ G.maybe x              -- Nothing : Just = 1 : 1 i.e.
equal ratio
```

==========

Usage

**Example**: Basic usage

```
{# LANGUAGE OverloadedStrings #}

import HedgeHog
import qualified HedgeHog.Gen as H
import qualified HedgeHog.Range as H
import Control.Exception
import Control.Monad

genNum :: Gen Int
genNum = H.int (H.linear 0 100)

runGroup :: IO ()
runGroup = do
  success <- checkSequential group
  unless success $ throw $ userError "FAIL!"

group :: Group
group = Group
  "Test"
  [
    ("reflexivity", property $ do
```

```
      x <- forAll genNum
      y <- forAll genNum
      x === x
      x /== y
    )
  ]
```

**GOTCHA**: **sample** function only generates small values
**sample** uses default **size 30** (see source code https://hackage.haskell.org/package/hedgehog-1.2/docs/src/Hedgehog.Internal.Gen.html#sample)
  e.g. if range is 0 ~ 100, size 30 reduces the range to (0 ~ 30)
Need to use **printWith** to specify a size

```
ghci> import Hedgehog
ghci> import Hedgehog.Gen
ghci> import Hedgehog.Range
ghci> import Hedgehog.Internal.Seed
ghci> gen = int (linear 1 100)

-- sample: uses size 30 i.e. range (1 ~ 30)
ghci> sample gen
11
ghci> sample gen
20

-- printWith: specify seed + size
ghci> printWith 99 (Seed 13 100) gen
=== Outcome ===
17
=== Shrinks ===
…

ghci> printWith 99 (Seed 13 100) gen            -- same seed, same
outcome
=== Outcome ===
17
=== Shrinks ===
…

ghci> printWith 99 (Seed 130 10) gen            -- different seed
=== Outcome ===
55
=== Shrinks ===
…

ghci> import Hedgehog.Internal.Seed
ghci> seed <- random                            -- random seed
ghci> printWith 99 seed gen
=== Outcome ===
60
=== Shrinks ===
```

```
1
…
```

Sample many with **replicateM**

```
-- Many with sample
ghci> replicateM 10 $ sample gen
[20,19,19,3,13,30,25,22,16,24]

-- Many with printWith
ghci> import Data.Foldable                    -- traverse_
ghci> seeds = replicateM 10 random            -- random seeds
ghci> join $ (traverse_ (\s -> printWith 99 s gen)) <$> seeds
=== Outcome ===
6
=== Shrinks ===
1
3
5
=== Outcome ===
56
=== Shrinks ===
1
28
….
```

Usage: print values used in tests
use **label/collect** to print all used values
e.g.

```
prop_shrinktest :: Property
prop_shrinktest = property $ do
  x <- forAll $ int $ linear 0 1000
  collect x
```

All values of x used

```
  ✓ prop_shrinktest passed 100 tests.
    0   1% |····················
    107 1% |····················
    110 1% |····················
    117 2% |····················
    119 1% |····················
    12  1% |····················
    121 1% |····················
    130 1% |····················
    136 1% |····················
    138 1% |····················
    139 1% |····················
```

```
    143 1% | ·····················
…
```

Usage: test coverage of generated numbers
Use **cover** function
e.g.

```
import Hedgehog
import Hedgehog.Gen
import Hedgehog.Range

prop_testcoverage :: Property
prop_testcoverage = property $ do
  x <- forAll $ int $ linearFrom 50 0 100
  cover 1 "[0,20)" $ 0 <= x && x < 20           -- at least 1% in
[0,20]
  cover 20 "[20,40)" $ 20 <= x && x < 40        -- at least 20% in
[20,40)
  cover 40 "[40,60)" $ 40 <= x && x < 60
  cover 20 "[60,80)" $ 60 <= x && x < 80
  cover 1 "[80,100)" $ 80 <= x && x < 100
```

output

```
-- e.g. success coverage
   ✓ prop_testcoverage passed 100 tests.
     [0,20)    6% ▉ ····················· ✓ 1%
     [20,40)  26% ▉▉▉▉▉ ················· ✓ 20%
     [40,60)  44% ▉▉▉▉▉▉▉▉▉ ············· ✓ 40%
     [60,80)  22% ▉▉▉▉ ·················· ✓ 20%
     [80,100)  2% ▌····················· ✓ 1%

-- e.g. failed coverage
  ✗ prop_testcoverage failed
    after 100 tests.
     [0,20)    5% ▉····················· ✓ 1%
  ⚠  [20,40)  19% ▉▉▉▉ ················· ✗ 20%
     [40,60)  50% ▉▉▉▉▉▉▉▉▉▉ ··········· ✓ 40%
  ⚠  [60,80)  18% ▉▉▉▉ ················· ✗ 20%
     [80,100)  8% ▉▉ ···················· ✓ 1%
```

Usage: **recursive** generators i.e. generator for nested values
Need to use **recursive** function since otherwise, may result in infinite nesting

```
recursive :: MonadGen m
  => ([m a] -> m a)
  -> [m a]                    non-recursive values
  -> [m a]                    recursive values
  -> m a
```

e.g. group of values

```
import Hedgehog
import Hedgehog.Gen as G
import Hedgehog.Range as R

data Value a =
   Value a
 | ValueGroup [Value a] deriving (Show, Eq)

singleValue :: Gen (Value Int)
singleValue = Value <$> int (linear 0 1000)

anyValue :: Gen (Value Int)
anyValue =
  recursive
    ((ValueGroup <$>) . subsequence <=< sequence)           --
create a ValueGroup from a subsequence of [Gen (Value Int)]
    (replicate 10 singleValue)                              -- non-
recursive values
    [anyValue]                                              --
recursive call of anyValue
```

e.g.

```
import GHC.Ext

prop_recursive :: Property
prop_recursive = property $ do
  let
    groupValues (ValueGroup v) = v
    groupValues _ = []
    groupDepth :: Value a -> Int
    groupDepth x = case x of
      Value _ -> 0
      ValueGroup v -> sum (groupDepth <$> v) + 1

  x <- forAll anyValue
  label (fromString $ "number of values in top level:" <> show
(length $ groupValue x))
  label (fromString $ "nesting depth:" <> show (groupDepth x))
```

output

```
  ✓ prop_recursive passed 100 tests.
    number of values in top level:1   2% ▏·····················
    number of values in top level:10  2% ▏·····················
    number of values in top level:2   2% ▏·····················
    number of values in top level:3   5% ▉·····················
    number of values in top level:4  18% ▉▉▉▏····················
    number of values in top level:5  25% ▉▉▉▉▉···················
    number of values in top level:6  25% ▉▉▉▉▉···················
```

```
    number of values in top level:7  13% ■■▌·················
    number of values in top level:8   7% ■▌··················
    number of values in top level:9   1% |··················
    max group depth:1                50% ■■■■■■■■■···········
    max group depth:2                21% ■■■■▌···············
    max group depth:3                17% ■■■▌················
    max group depth:4                 8% ■▌··················
    max group depth:5                 1% |··················
    max group depth:6                 2% ▌··················
    max group depth:7                 1% |··················
```

Usage**:** auto generate group
Use **discover/discoverPrefix**.
Assumes "prop_" prefix by default

```
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE TemplateHaskell #-}

import Hedgehog
import Hedgehog.Gen as G
import Hedgehog.Range as R

main :: IO ()
main = do
  testSuccess <- checkSequential allTests

allTests :: Group
allTests = $$discover
allTests = $$(discoverPrefix "prop")           -- same

prop_a :: Property
prop_a = property $ do
   ...

prop_b :: Property
prop_b = property $ do
   ...
```

output

```
── Main ──
  ✓ prop_a passed 100 tests.
  ✓ prop_b passed 100 tests.
```